

Traitement automatique du langage en python : Exemple de la navigation par affinités dans l'Encyclopædia Universalis

Présentation de l'entreprise.....	2
Idee de départ	5
Description du projet	6
Justification pour le DVD Universalis.....	7
Justification pour le .fr.....	7
Justification pour les sites institutionnels	8
Un peu de théorie.....	9
Tokenisation	9
Mots vides	9
Lemmatisation / Racinisation.....	10
TfIdf	11
Calcul de similarité cosinusoidale.....	12
Collocation	12
Hapax	12
Présentation des bibliothèques.....	13
Python.....	13
NLTK.....	13
Gensim.....	14
Raphaël.JS.....	15
Évaluation des résultats	17
Précision et Rappel.....	17
Évaluation par nombre d'entrées d'index commune.....	17
AB Testing.....	18
Bilan et Perspectives	20

Présentation de l'entreprise

Encyclopædia Universalis est une maison d'édition spécialisée dans l'édition de ressources encyclopédiques. Elle a été créée en 1966 conjointement par le Club Français du Livre et Encyclopædia Britannica. La première version de l'encyclopédie voit le jour en 1968 sous la direction de Claude Grégory. Depuis, une nouvelle édition a été produite environ tous les cinq ans jusqu'à celle de 2011, annoncée comme la dernière version papier et qui a marqué la fin de ce type de diffusion.

En 1995, l'encyclopédie est éditée sous la forme d'un CD-ROM, puis de DVD-ROM. Les versions numériques paraissent en septembre de chaque année millésimée de l'année suivante.

En parallèle de la version sur DVD-ROM, une version en ligne est disponible. Celle-ci est mise à jour tous les 3 mois. La version en ligne est accessible au grand public par abonnement. Une version spécifique à destination du public institutionnel est également disponible. Ces versions on-line permettent de lutter contre la baisse du chiffre d'affaires due à la baisse des ventes de la version DVD.

Ces trois supports contiennent à peu de choses près le même contenu éditorial composé à l'heure où j'écris ces lignes de plus de 33 000 articles, 9 900 photos achetées auprès d'agences et plus de 11 000 documents multimédias réalisés par le service multimédia.

Ces documents (articles et images) sont tous indexés manuellement par des mots clefs appelés « entrées d'index » et classifiés dans une arborescence d'environ 10 000 catégories. Ils sont également indexés par le moteur de recherche Lucène pour permettre une recherche « full-text ». Les deux types de recherche sont proposés sur les différents supports numériques.

L'âge d'or des encyclopédies papier concerne principalement les années 1970 à 2000. Les ventes de la collection papier d'Universalis se chiffraient à environ 20 000 exemplaires par an. À cette époque, l'entreprise a pour principaux concurrents les éditions Larousse avec son Grand Larousse encyclopédique en 10 volumes, mais aussi

d'autres éditeurs qui produisent des ouvrages de référence dans différents domaines, comme les Presses Universitaires de France pour la collection « Que sais-je ? » ou le Quid pour des informations grand public.

Les années 90 ont marqué le passage au CD-ROM pour les différents acteurs du marché. Microsoft a édité son encyclopédie Encarta à partir de 1993, Universalis a lancé sa première édition en 1995, Hachette Multimédia a sorti son CD-ROM en 1999. Aujourd'hui, il ne reste sur le marché que la version de l'encyclopédie Universalis. Cette longévité s'explique d'une part par le positionnement haut de gamme, les articles sont écrits par un expert reconnu dans leur domaine. D'autre part, par le système de vente en option négative, par lequel, les clients peuvent tester gratuitement la nouvelle version du logiciel dès sa sortie au mois de septembre.

Par la suite, les années 2000 ont été marquées par l'accès à Internet à grande échelle. La version en ligne d'Universalis date de 1999 d'abord pour le public institutionnel avant le lancement de sa version pour le grand public. Au niveau de la concurrence, on peut citer Hachette et Encarta qui ont également eu une version en ligne. En 2001, Jimmy Wales a lancé Wikipédia qui repose sur une rédaction collaborative et bénévole des articles. Très vite, Wikipédia s'est forgé une notoriété importante auprès du grand public qui pouvait obtenir gratuitement l'information. Wikipédia garde néanmoins une image d'encyclopédie dont les informations peuvent être soumises à polémique par la facilité d'édition des articles. Le corps enseignant est souvent partagé sur la question et apprécie d'avoir un contenu pédagogique qui reste stable et fiable à travers le temps.

Pour ces raisons, Universalis focalise son approche commerciale sur le monde institutionnel. L'entreprise est du fait, présentée de plus en plus comme un concurrent des éditeurs de manuel scolaire. En effet, le ministère de l'éducation nationale souhaite aujourd'hui voir dématérialisés les manuels scolaires. Leur utilisation est de plus en plus remise en cause par les enseignants au profit de ressources pédagogiques en ligne.

Enfin, il est à noter qu'Universalis s'est lancé depuis deux ans sur la commercialisation

de livres électroniques en partenariat avec Amazon. Ce nouveau marché permet de commercialiser un article ou un ensemble restreint d'articles sur un thème donné. Il vise le grand public (même si les lycéens ou les étudiants sont des clients fréquents, ils le sont cette fois-ci à titre individuel) et les ventes restent marginales.

Idée de départ

Au départ du projet, il y avait une idée un peu farfelue qui consistait à rechercher des solutions pour indexer automatiquement les articles de l'encyclopédie par des entrées d'index de notre base.

Le service indexation s'occupe d'indexer et de poser les balises dans les textes pour l'ensemble des articles de l'encyclopédie. Ce processus supplémentaire impacte d'autant le planning des livraisons et ne peut pas être effectué en parallèle d'autres étapes.

Différentes idées ont été soulevées comme un filtrage bayésien à l'image de la classification des mails de spam ou bien une recherche plein texte de l'ensemble des libellés d'entrées d'index par le moteur Lucène.

Au fur et à mesure des essais, les résultats les plus prometteurs étaient fournis par l'analyse des entrées d'index contenu dans les plus proches voisins des documents. La distance des documents les uns par rapport aux autres est calculée par similarité cosinusoidale (voir Calcul de similarité p.12). Les entrées d'index de ces voisins ont une probabilité d'être pertinentes dans le document en question d'autant plus élevée que le voisin en question est proche. La similarité ayant une valeur comprise entre 0 et 1 (1 signifie que les deux documents sont exactement au même endroit), il suffit d'affecter cette valeur à chaque entrée d'index des voisins et d'en faire la somme.

Description du projet

La recherche de similarité d'articles peut être utilisée dans différents projets. Par exemple, il est possible d'utiliser la similarité cosinusoidale pour pouvoir afficher les articles « proches » de celui consulté afin d'inciter le lecteur à naviguer à travers d'autres articles de l'encyclopédie.

Cette navigation a été appelée « Affinité ». Elle consiste en l'affichage pour chaque article de l'encyclopédie des 6 plus proches voisins et pour chacun de ces voisins, l'affichage de 2 voisins supplémentaires si ceux-ci ne sont pas déjà affichés dans le voisinage.

Ce qui permet d'atteindre 18 articles relativement proches sous forme d'un réseau avec l'article principal au centre. Pour chaque nœud du réseau, il est possible soit d'afficher l'article en question, soit de placer le nœud au centre de l'écran pour pouvoir continuer à naviguer un peu plus loin.

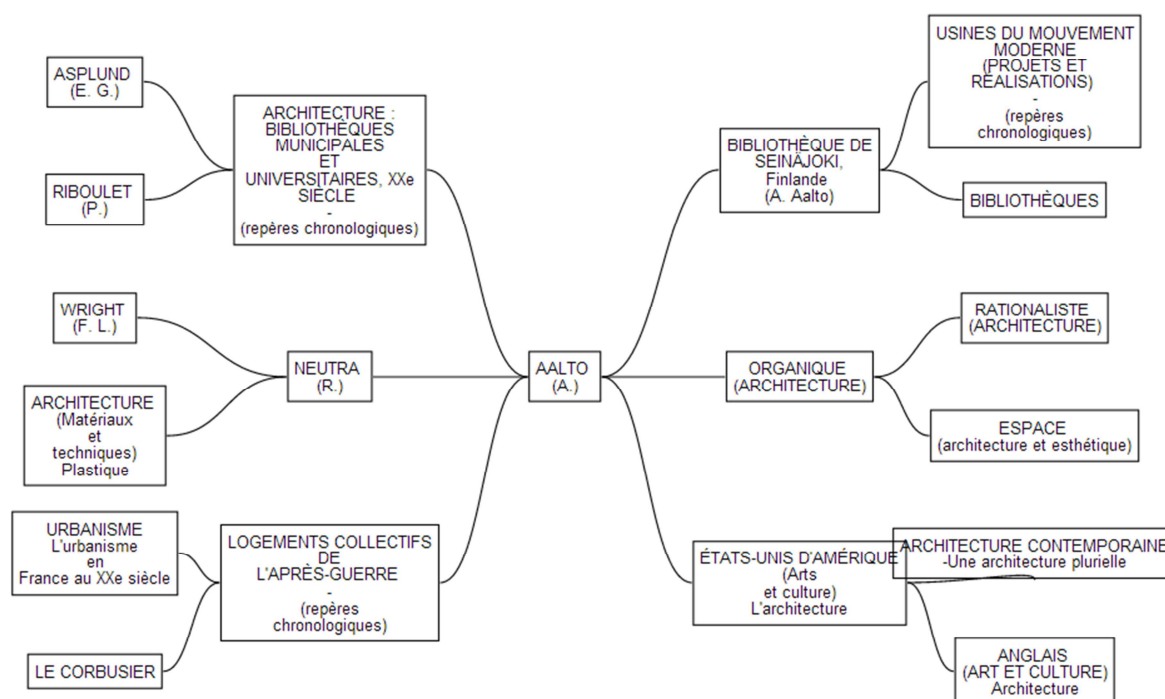


Figure 1 – Premier essai de navigation par Affinité à partir de l'article sur l'architecte finlandais "A. Aalto" (version utilisée actuellement dans l'intranet interne à l'entreprise)

Justification pour le DVD Universalis

Tous les ans, un nouveau DVD est élaboré pour les clients d'Universalis. Certains articles sont mis à jour, de nouveaux articles sont ajoutés. Ce DVD est soumis à l'évaluation des clients qui choisissent ensuite de l'acheter ou de le renvoyer.

Il est donc important pour l'entreprise de proposer tous les ans un produit contenant suffisamment de nouveauté pour justifier l'achat auprès de ses clients.

L'ajout de cette navigation par « affinité » à l'avantage d'être visible sur chacune des pages articles et permet aux lecteurs de prolonger leur navigation au sein du DVD.

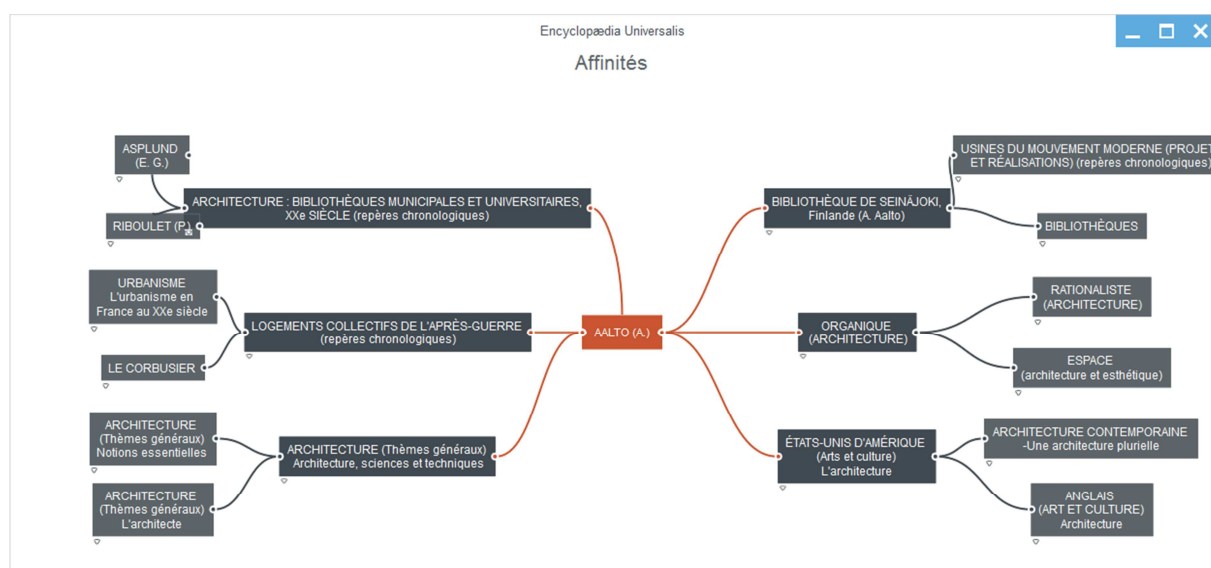


Figure 2 - Affichage de la navigation par affinité utilisée sur le DVD pour l'article "A. Aalto"

Justification pour le .fr

Sur le site internet grand public, les besoins sont un peu différents. Le principal trafic de l'encyclopédie vient directement de Google. Ainsi, pour inciter les visiteurs à prendre un abonnement annuel au contenu de l'encyclopédie, nous proposons gratuitement l'accès au début de nos articles. Ce contenu est indexé par les robots de Google. À l'aide de l'algorithme de Page Rank, Google affiche en tête de résultat une page si celle-ci contient beaucoup de liens entrants. On recherche donc à augmenter le nombre de liens entre les articles de l'encyclopédie. La navigation par « affinité » permet donc des nouveaux liens entre pages et améliore la visibilité de nos articles sur le moteur de recherche Google.



Figure 3 - Élément graphique du site internet pour ouvrir la navigation par affinité

Justification pour les sites institutionnels

Les sites institutionnels ont la particularité d'être achetés par des conseils régionaux (pour les lycées), des conseils généraux (pour les collèges) ou d'autres structures (pour les universités). Les personnes qui les achètent ne sont pas les mêmes personnes que ceux qui les utilisent. Il est donc indispensable pour justifier l'achat de fournir des statistiques de consultation des différents sites (nombre de pages vues, nombre d'utilisateurs différents, évolution des consultations dans le temps, principales pages consultées, etc.)

L'ajout de la navigation par « affinité » permet d'augmenter la consultation des sites et d'améliorer les statistiques.

Un peu de théorie

Avant de pouvoir faire le calcul de distance des articles proprement dit, les documents subissent différents traitements préparatoires.



Tokenisation

Le découpage des articles en mot est un processus qui ouvre différentes possibilités. Faut-il considérer « Aujourd'hui » comme deux mots différents ou un seul mot ? Faut-il découper suivant les espaces et les signes de ponctuation ?

La solution retenue est le découpage par une expression rationnelle :

```
tok2 = nltk.RegexpTokenizer(r'\'\'(?x)
    \d+(\.\d+)?\s*%    # les pourcentages
    | \w'              # les contractions d', l', ...
    | \w+              # les mots pleins
    | [^\w\s]          # les ponctuations
    '\')
```

En utilisant un découpage par la bibliothèque NLTK (voir NLTK p.13).

Mots vides

Les mots vides sont des mots très fréquents qui apportent peu d'informations. Il s'agit des pronoms personnels, des adjectifs possessifs, certaines prépositions, etc. Supprimer ces mots vides n'a pas d'impact sur les résultats et permet d'obtenir des gains en termes d'utilisation de la mémoire et de temps processeurs lors du traitement. Néanmoins, il convient de les choisir judicieusement pour être sûr que leur suppression n'ait vraiment aucun impact sur les résultats. Une analyse rapide du corpus (l'ensemble des documents) permet d'extraire la liste des termes qui sont présents plus de 10 000 fois.

Nous obtenons ainsi une liste de 175 mots vides que nous pouvons supprimer des articles.

Lemmatisation / Racinisation

La définition de Wikipédia donne :

La lemmatisation désigne l'analyse lexicale du contenu d'un texte regroupant les mots d'une même famille. Chacun des mots d'un contenu se trouve ainsi réduit en une entité appelée lemme (forme canonique). La lemmatisation regroupe les différentes formes que peut revêtir un mot, soit : le nom, le pluriel, le verbe à l'infinitif, etc.

On peut donner comme exemple « Fleuriste », « floral », « fleurs » qui ont comme lemme commun « fleur ».

D'autre part, la racinisation consiste à obtenir une « racine ». À la différence avec le lemme, la racine ne donne pas toujours un mot du dictionnaire. On peut obtenir à partir de « parler » la racine « parl » alors que son lemme est « parole ».

Néanmoins, la lemmatisation et la racinisation ont le même objectif : pouvoir traiter deux mots de la même famille comme étant porteur du même sens et leur considérer la même importance.

La première approche que j'ai eu pour traiter cette étape est d'utiliser une lemmatisation stricte, c'est-à-dire que pour chaque terme, il me semblait souhaitable d'avoir le mot exact dont était issu le terme. J'ai donc utilisé le *lexique des formes fléchies du français* fourni par l'INRIA (Institut National de Recherche en Informatique et en Automatique). Ce document contient 500 000 lignes avec à chaque fois la forme fléchie, le lemme et le rôle grammatical du mot (adjectif, verbe, etc.)

Exemple :

pendant adj [pred='pendant____1<subj:(sn),obl:(de-sn de-sinf de-scompl à-sn à-sinf à-scompl)>',cat=adj,@ms]
pendant v [pred='pendre____1<subj:sn sinf scompl,obj:(sn)>',cat=v,@G]

On s'aperçoit très vite que d'une part, le chargement en mémoire de ces données est relativement coûteux en temps (le fichier fait 41 Mo) et que le résultat n'est pas si bon que ça. En effet, un grand nombre de synonymes génère des incertitudes. « Pendant » est-il une préposition (pendant ce temps) ou bien le participe présent du verbe « pendre » ?

La deuxième approche testée consiste à utiliser l'algorithme de Potter. Cette approche purement logicielle consiste à retirer les lettres de la fin du mot en fonction d'une cinquantaine de règles appliquées en 7 phases.

Le résultat est rapide à obtenir, grâce à la bibliothèque NLTK (voir p.13).

Nous utilisons l'appel suivant :

```
stemmatiser = FrenchStemmer()  
def stemmer(mot) :  
    return stemmatiser.stem(mot)
```

Et nous l'appelons sur une liste python de mot comme ceci :

```
mots = [stemmer(mot) for mot in mots]
```

Nous obtenons par exemple la transformation suivante :

Texte original (début de l'article sur l'architecte finlandais Alvar Aalto)

Le Finlandais Alvar Aalto, comme le Suédois Gunnar Asplund, figure parmi les rares architectes scandinaves qui ont acquis une notoriété internationale avant la Seconde Guerre mondiale.

Transformation par suppression de mots vides et racinisation par l'algorithme de Potter :

finland / alvar / aalto / comm / suédois / gunnar / asplund / figur / parm / architect / scandinav / qui / ont / international / avant / second / guerr / mondial

TfIdf

Le TfIdf (parfois noté tf*idf) est une méthode de pondération des mots en recherche d'information. Elle se base sur le Tf : Term Frequency (fréquence du terme) et sur l'Idf : Inverse Document Frequency (fréquence inverse du document).

L'idée sous-jacente est qu'un terme est d'autant plus important qu'il est fréquent dans un document donné ET rare dans les autres documents.

Nous avons les formules suivantes :

$$Tf(t,d) = \frac{n_{t,d}}{\sum_t n_{t',d}}$$

Avec $n_{t,d}$ correspond au nombre d'occurrences de t dans d

$$\text{Idf} = \log \frac{|D|}{|\{d' \in D \mid n_{t,d'} > 0\}|}$$

Calcul de similarité cosinusoidale

La similarité cosinusoidale est une valeur comprise entre 0 et 1 correspondant à la similarité de deux documents. Les documents sont représentés sous forme de vecteur dans un espace à n dimensions (où n est le nombre de mots différents de vocabulaire utilisé). Sur chacune des dimensions, la valeur du Tfidf représente le poids du mot en question. Nous obtenons ainsi des vecteurs qui ont beaucoup de dimension, mais dont la plupart des axes ont un poids à zéro (car tous les mots du dictionnaire ne se trouvent pas dans chaque article. La distance est ensuite calculée par le cosinus de l'angle entre ces deux vecteurs. Il s'agit de calculer le produit scalaire des deux vecteurs après les avoir normalisés. Cela se fait en multipliant deux à deux les Tfidf des termes communs aux deux vecteurs et à les cumuler.

Mais en pratique, la bibliothèque Gensim s'occupe de tous ces calculs (voir Gensim p.14)

Collocation

En linguistique, une collocation (qui signifie « placer ensemble ») est un ensemble de mot dont la fréquence d'apparition commune est très élevée et pour lesquels le sens des mots ensemble est différent de celui pris séparément.

Par exemple « pomme de terre » n'a que peu de lien avec les mots « pomme » et « terre ».

La détection des collocations devrait permettre d'obtenir de l'information supplémentaire en analysant leur fréquence.

Dans le projet actuel, les collocations ne sont pas prises en compte.

Hapax

Les Hapax sont des mots qui n'apparaissent qu'une seule fois dans l'ensemble du corpus. Par ce fait, leur Tfidf est très élevé, mais par définition, ils ne peuvent pas augmenter la similarité d'un autre document puisqu'ils ne se trouvent dans aucun autre document.

Ils peuvent néanmoins être listés pour pouvoir détecter d'éventuelle coquille orthographique.

Présentation des bibliothèques

Python

Python est un langage de programmation Open Source qui a été créé en 1990. Il est multiplateforme, objet, interprété et possède un grand nombre de bibliothèques qui lui permettent d'effectuer un grand nombre de tâches (lecture/écriture de document Excel, accès aux bases de données, manipulation par DCOM d'applications clientes, serveur web, etc.)

Nous utilisons Python pour sa simplicité d'utilisation et sa capacité à apporter une solution à un éventail très large de problèmes.

Pour pouvoir prototyper rapidement des pages internet, nous utilisons PSP (Python Server Pages) qui est un module apache et qui permet d'intégrer des scripts python dans une page très facilement.

Site Internet : <http://modpython.org/>

NLTK

Natural Language Toolkit est une bibliothèque Python permettant le traitement automatique des langues. Elle contient des échantillons de corpus dans différentes langues (une partie de la Bible, des discours politiques, des œuvres de Shakespeare, des extraits de conversation internet issue de « chatrooms »). Les outils fournis avec cette bibliothèque permettent d'effectuer de l'analyse grammaticale des mots, de les représenter graphiquement (avec l'utilisation de la bibliothèque Matplotlib), d'effectuer un alignement des mots pour voir dans quels cas de figure une expression est utilisée, de faire du découpage de mot, de la lemmatisation, etc.

Nous utilisons cette bibliothèque pour la lemmatisation (voir p.10) et pour le découpage des mots (voir Tokenisation p.9)

Bibliographie : *Natural Language Processing with Python* de Steven Bird, Ewan Klein et Edward Loper

Site Internet : <http://www.nltk.org/>

Gensim

Gensim est une bibliothèque open source, écrite en python, permettant le calcul de Topic model dans un modèle vectoriel. Elle repose sur les bibliothèques Scipy et Numpy, deux bibliothèques python permettant les calculs matriciels sur un grand nombre de dimensions.

Elle implémente plusieurs algorithmes de modèle à espace vectoriel, comme le LDA (Allocation de Dirichlet Latente), LSI (Latent Semantic Indexing), le HDP (Hierarchical Dirichlet Process) et le TfIdf (voir p.11)

Gensim a la particularité d'être indépendante de l'espace mémoire. C'est-à-dire que l'ensemble du corpus n'est pas chargé en mémoire et utilise l'espace disque pour faire des calculs. En théorie, il est tout à fait possible de traiter des quantités très importantes de données, les exemples cités dans le tutoriel sur le site de la bibliothèque montrent des exemples d'utilisation sur le corpus de l'encyclopédie Wikipédia en langue anglaise (44 Go de données décompressées)

En pratique, il m'est arrivé à plusieurs reprises d'avoir des dépassements de mémoire en utilisant certains modèles algorithmiques. Il convient d'être prudent quand on manipule des quantités de données de cette taille pour essayer d'être économe en mémoire et en temps CPU. Une des solutions pour économiser de la mémoire est de limiter la taille du dictionnaire (en supprimer des mots vides). Une autre solution est d'utiliser une architecture 64bits qui permet un plus grand adressage de mémoire.

Le code principal pour le calcul de similarité est le suivant :

```
dictionary = corpora.Dictionary(getAllDocs())
corpus = [dictionary.doc2bow(texte) for texte in getAllDocs()]
tfidf = models.TfidfModel(corpus)
index = similarities.SparseMatrixSimilarity(tfidf[corpus],
num_features=len(dictionary))
for numref in dictId2Numref.values() :
    doc = getDoc(str(numref))
    vec_bow = dictionary.doc2bow(doc)
    sims = index[tfidf[vec_bow]]
    sims = sorted(enumerate(sims), key=lambda item: -item[1])
```

getAllDocs() est un itérateur qui fournit l'ensemble des documents après traitements (tokenisation, suppression des mots vides, lemmatisation)

Dans un premier temps, nous calculons le dictionnaire (c'est-à-dire que nous listons l'ensemble des mots du corpus).

Le Corpus est ensuite mis dans une matrice de « Sac de Mot » (Bag of Words : BOW)

Le modèle vectoriel utilisant le TfIdf est calculé ainsi qu'une matrice de similarité. Nous reprenons ensuite chacun des articles du corpus pour l'évaluer par rapport à cette matrice de similarité. Nous obtenons une liste de résultat du document en question et de l'ensemble des autres documents avec leur cosinus respectif (voir Calcul de similarité cosinusoïdale p.12).

Le site internet de la bibliothèque contient un tutoriel assez bien fait pour apprendre à utiliser la bibliothèque. Même si jusqu'à présent, il m'a été relativement difficile de « voir ce qu'il y avait sous le capot » et d'essayer de mieux analyser le comportement de cette bibliothèque. Les seules marges de manœuvre consistaient à tester différents paramètres et différents modèles vectoriels et de faire une évaluation des résultats (voir p.17)

Actuellement, nous utilisons le modèle TfIdf qui permet de calculer la matrice de toutes les similarités des 33 000 articles de notre corpus en 40 minutes environ sur une machine i5 avec 8Go de RAM.

Site Internet : <http://radimrehurek.com/gensim/>

Raphaël.JS

Raphaël.JS est une bibliothèque Javascript permettant d'écrire du code SVG (Scalable Vector Graphics) sur un canvas html5.

J'ai testé plusieurs bibliothèques pour visualiser le résultat et il était difficile de concilier l'affichage d'un grand nombre de données avec une interface utilisateur claire et facile à manipuler.

L'utilisation de la bibliothèque Raphaël.JS permet de manipuler des éléments graphiques de bas niveau (rectangle, cercle, ligne, etc.) et d'en avoir une parfaite maîtrise pour l'utilisation que l'on souhaitait en faire. Le choix de n'afficher que les 6 plus proches voisins d'un article nous a permis de garder une interface qui ne soit pas trop surchargée. Certains articles ayant des noms assez longs, un nombre plus important de titres affichés à l'écran rendait le résultat illisible.

L'utilisation du SVG a permis ainsi d'avoir une interface épurée et agréable : utilisation de rectangle aux coins arrondis, de courbe de bézier pour lier les articles entre eux, de dégradés, etc.

Nous avons également utilisé du CSS pour améliorer l'affichage des objets SVG.

Voici un exemple d'utilisation de cette bibliothèque :

```
var t_titre = paper.text(delta_largeur+(largeur/2),hauteur/2,
titre).attr({"font-family": "Arial", "font-size": 12, 'fill':'#ffffff' });
t_titre.data("numref",nref);
t_titre.node.id = nref;
var c_titre = paper.rect(t_titre.getBBox().x-10,t_titre.getBBox().y-10,
t_titre.getBBox().width+20, t_titre.getBBox().height+20, 0).attr({stroke:
'#c95530','stroke-width': 1, 'fill': '#c95530'}).toBack();
var button_titre = paper.set().push(c_titre).push(t_titre);
```

Les objets SVG reçoivent des attributs CSS de remplissage ou pour définir les polices de caractères utilisés. C'est une liberté très importante qu'on ne retrouve pas dans d'autres bibliothèques d'affichage de données que nous avons essayé.

Nous utilisons des appels AJAX pour pouvoir recalculer l'arbre des voisins à partir d'une nouvelle position. Ces appels permettent d'avoir une ergonomie rapide de navigation.

Nous utilisons également un objet javascript afin de simuler le fonctionnement de l'historique de navigation. Ainsi, il est possible de revenir facilement en arrière sur un article déjà consulté.

Site Internet : <http://raphaeljs.com/>

Évaluation des résultats

Précision et Rappel

Habituellement dans la recherche d'Information, on utilise deux critères d'évaluation des résultats : la précision et le rappel

La précision est le nombre de documents trouvés pertinents par rapport au nombre de documents total proposé par l'algorithme.

Le rappel est le nombre de documents trouvés par rapport au nombre de documents pertinents.

Il est difficile d'utiliser ces critères dans notre cas de figure, car nous souhaitons, pour des raisons d'interface et d'ergonomie n'afficher « que » 6 documents, quelque-soit le document de départ. D'autre part, les faux positifs que l'on peut trouver sont généralement éliminés par le filtrage par « entrée d'index commune ». En effet, deux articles ayant une entrée d'index commune évoqueront forcément un thème commun sur le sujet de cette entrée d'index.

Évaluation par nombre d'entrées d'index commune

Les différentes étapes du processus imposent le choix de certains paramètres (mots vides choisis, règles de lemmatisation, etc.). Et chacun de ces paramètres va générer un résultat différent de l'ensemble des voisins d'un article donné.

Il est important d'avoir des critères d'évaluation du résultat obtenu afin de savoir si on se rapproche ou au contraire on s'éloigne de ce que l'on souhaite obtenir.

Pour notre projet de lister les 6 articles les plus proches d'un article donné, il est difficile d'évaluer si les résultats sont bons ou mauvais, du moins pas de manière automatique. Quand on voit l'article "Jeanne d'Arc" qui apparait comme voisin de l'article "Jeanne Moreau", on comprend bien que ces deux articles partagent le mot "Jeanne" qui est fréquent dans ces deux textes et relativement rare au sein du corpus. Il est donc tout à fait compréhensible que ces deux articles aient une similarité cosinusoidale élevée.

Deux articles proches partagent des métadonnées communes. Nous avons pour les articles en base un système d'indexation manuel. Le service indexation est chargé de placer dans les textes des balises parmi un ensemble de 70 000 entrées d'index. Il est

fréquent que deux articles "voisins" aient des entrées d'index communes. Nous utiliserons donc ce critère pour évaluer le résultat du calcul effectué par la bibliothèque Gensim en se basant sur un jeu de 250 articles choisis pour couvrir différentes disciplines de l'encyclopédie.

Pour chacun de ces 250 articles, nous retenons les 6 articles les plus proches et effectuons la somme du nombre d'entrées d'index communes aux 6 articles par rapport à l'article concerné.

Par exemple, nous avons :

Identifiant	Intitulé	Taux (cos)	Nb Balise index
<u>L110701</u>	LOUIS XIV	1.0	47
<u>H980021</u>	FRANCE (Histoire et institutions) - Formation territoriale	0.533127	12
<u>C933041</u>	BOURBONS	0.398898	7
<u>MN01018</u>	FRANCE : LA POLITIQUE D'EXPANSION SOUS LOUIS XIV - (repères chronologiques)	0.471971	5
<u>T314029</u>	TORCY (marquis de)	0.306272	7
<u>Z020031</u>	ANNEXION DE STRASBOURG PAR LOUIS XIV	0.488441	4
<u>H980261</u>	FRONDE	0.388773	5

Pour l'article Louis XIV, l'ensemble de ses balises index est commun avec lui-même et son taux de similarité est de 1 (cosinus de l'angle 0°). La somme du nombre des entrées d'index est de 40 (12+7+5+7+4+5).

Cela permet d'évaluer les différents paramètres de l'algorithme qui donne un taux de similarité.

AB Testing

Un autre domaine où il est important d'évaluer les résultats concerne le site internet. L'utilisation du bouton d'ouverture de la page "Navigation par Affinité" peut être placée en haut de l'article, en bas, suivant certaines tailles. Pour utiliser au mieux cette fonctionnalité, nous faisons ce qu'on appelle un AB testing. C'est-à-dire que nous utilisons un système qui propose aux clients alternativement deux solutions (l'une appelée A et l'autre appelée B), et nous analysons le nombre de personnes qui cliquent sur le bouton suivant l'un des deux cas de figure.

Cette solution permet d'évaluer la pertinence d'un bouton, d'une mise en page, mais aussi le tarif d'un abonnement (c'est ainsi que l'on a baissé l'abonnement annuel au site d'Universalis de 89 euros à 39 euros)

Bilan et Perspectives

Les différents essais autour du calcul de similarité entre deux textes ont permis de développer la recherche de « Navigation par Affinité » sur le DVD. Cette fonctionnalité a ensuite été reprise sur les différents sites de l'encyclopédie Universalis (site public et site institutionnel). Cette innovation sur le DVD est en partie responsable des ventes du DVD, car aujourd'hui, très peu de nouveaux clients achètent le DVD et le marché est principalement constitué des clients qui achètent une mise à jour et à qui il est nécessaire de présenter tous les ans de nouvelles fonctionnalités.

Il existe néanmoins d'autres pistes à creuser pour pouvoir améliorer le fonctionnement. Nous avons par exemple filtré les résultats donnés par les « entrées d'index communes », c'est-à-dire que nous n'affichons comme voisin que les articles qui possèdent au moins une entrée d'index en commun avec l'article traité.

On peut également découper les articles en documents plus petits (il peut exister jusqu'à 4 niveaux d'intertitre). Il est possible aussi de faire des recherches au niveau des collocations (N-gram) qui pourraient donner des résultats de meilleure qualité.

Cette technique de calcul de similarité peut être utilisée dans d'autres domaines que les outils publics. Afin d'aider les éditeurs et le service d'indexation, il est tout à fait possible d'utiliser cette solution pour choisir des entrées d'index (voir Idée de départ p.5) ou pour choisir des catégories dans lesquelles ranger l'article en question. Il est également possible d'utiliser cette solution pour proposer des « corrélats » qui sont des articles dont on conseille la lecture pour approfondir le sujet traité par un article donné (généralement à la fin de l'article sous la mention « voir aussi »)

Une autre façon d'utiliser la proximité des articles est de créer des petits groupes d'articles autour d'un domaine précis pour les commercialiser sous forme d'ebook. Le modèle économique des ebooks repose sur la longue traîne, c'est-à-dire une offre très importante de livres différents qui généreront chacun quelques ventes. Un outil pour sélectionner rapidement un grand nombre de groupe d'articles permet ainsi d'enrichir facilement et rapidement le catalogue.